

HW4 — Deep Reinforcement Learning Survey

Survey of DRL, Agentic AI, and SOTA Applications

May 2026

Outline

1. **Introduction to DRL** — Fundamentals + 10 Algorithms
2. **DRL Platforms** — Isaac Gym, SB3, CARLA, and more
3. **Agentic AI** — RLHF, LLM Agents, Multi-agent Systems
4. **Applications** — Robotics, Game AI, FinTech, Autonomous Vehicles
5. **SOTA Trends 2025–2026** — World Models, Diffusion Policy, Embodied AI
6. **Comparative Analysis** — DQN / PPO / SAC / MuZero / Decision Transformer
7. **Open Source Ecosystem** — GitHub Survey
8. **Bonus Experiments** — Our Results

Part 1 — What is Reinforcement Learning?

An agent learns by interacting with an environment to maximize cumulative reward

State s_t → Agent → Action a_t → Environment → Reward r_t → State s_{t+1}

MDP Tuple: (S, A, P, R, γ)

Component	Meaning
S	State space
A	Action space
$P(s' s, a)$	Transition dynamics
$R(s, a)$	Reward function
γ	Discount factor

Key challenge: Exploration vs. Exploitation

Part 1 — 10 DRL Algorithms Overview

Algorithm	Type	Action Space	Key Innovation
DQN	Off-policy	Discrete	Replay buffer + target network
Double DQN	Off-policy	Discrete	Decouple selection & evaluation
PPO	On-policy	Both	Clipped surrogate objective
A2C/A3C	On-policy	Both	Advantage actor-critic
SAC	Off-policy	Continuous	Maximum entropy + auto- α
TD3	Off-policy	Continuous	Twin critics + delayed update
MuZero	Model-based	Both	Learned world model + MCTS
Decision Transformer	Offline	Both	RL as sequence modeling
Offline RL (CQL)	Offline	Both	Conservative Q-value learning
Hierarchical RL	Any	Any	Sub-goal decomposition

Part 1 — DQN vs PPO (Our Experiment)

Bonus Experiment 1: CartPole-v1, 60k steps each

- **PPO**: Converges to perfect score (500) within 150 episodes — **stable monotonic improvement**
- **DQN**: Peaks at ~471 around episode 300, then **catastrophically forgets**
- Same phenomenon seen in RLHF: PPO preferred for stability

Part 1 — SAC: Maximum Entropy Framework

Why SAC beats DQN on continuous control:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_t r_t + \alpha \cdot \mathcal{H}(\pi(\cdot|s_t)) \right]$$

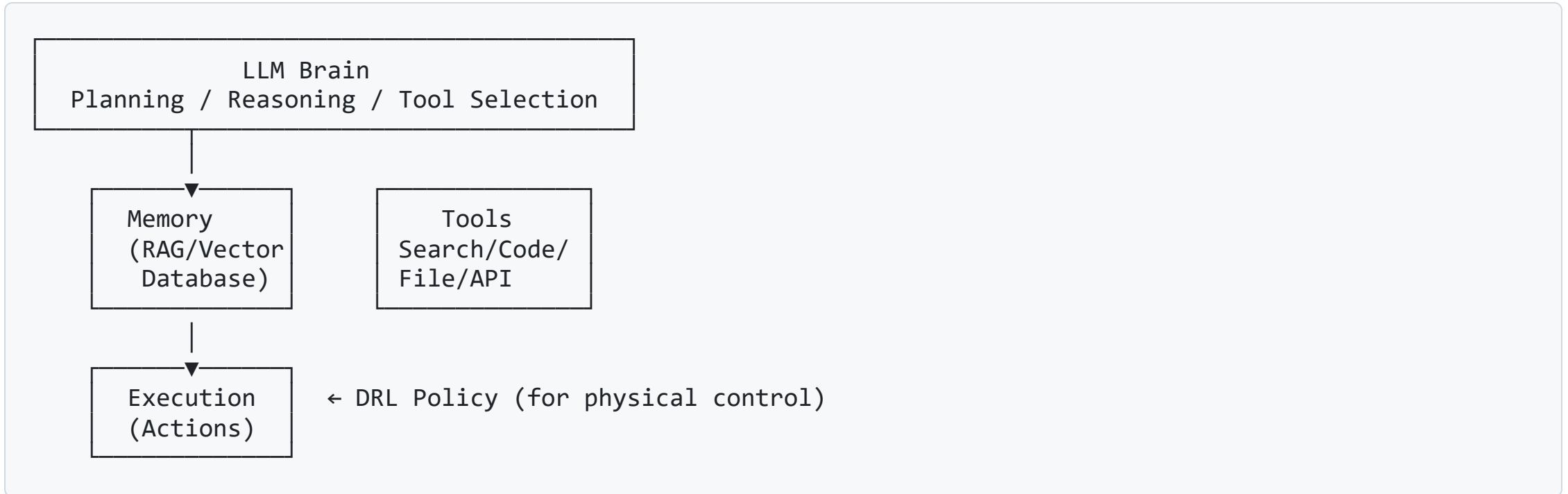
Feature	DQN	SAC
Action space	Discrete only	Continuous
Exploration	ϵ -greedy (manual)	Entropy (automatic α)
Sample efficiency	Low	High
LunarLander	✗ Cannot use	✓ 286.8 reward

Bonus 5 Result: SAC achieves **286.8 ± 16.7** on LunarLanderContinuous-v3 (threshold: 200)

Part 2 — DRL Platforms Comparison

Platform	Focus	Parallelism	Key Advantage
Isaac Gym	Robot locomotion	8,192 envs/GPU	Fastest physics simulation
CARLA	Autonomous driving	Low	Photorealistic + full sensor suite
Habitat Lab	Indoor navigation	64-512 GPUs	Largest indoor scene dataset
Unity ML-Agents	General games	Medium	Artist-friendly design
RLlib	Distributed RL	1000s of CPUs	Production-scale
SB3	Research	Single machine	Reliability + documentation
FinRL	Finance	Single machine	Financial data integration
MineDojo	Open-world	GPU clusters	Internet-scale training data

Part 3 — Agentic AI Architecture



DRL's role in agents: Action generation, reward optimization, physical control

Part 3 — RLHF / RLAIIF Pipeline

How ChatGPT and Claude are trained:

1. SFT : LLM trained on human demonstrations
2. RM : Reward model from human pairwise preferences
3. PPO : LLM optimized against RM (+ KL penalty)

RLAIIF (Constitutional AI): Replace human annotators with AI judge

- More scalable (no human annotation cost)
- More consistent (same principles applied uniformly)
- Enables continuous self-improvement

Key Risk: Reward hacking — model games the reward model without genuinely improving

Part 3 — Key Agentic Systems

System	Core Tech	Key Capability
Voyager	GPT-4 + Minecraft	Lifelong skill accumulation, code-based skills
OpenAI Deep Research	GPT-4o + web search	Multi-step autonomous research reports
RT-2	Vision-language-action	Generalist robot manipulation
AutoGPT	GPT-4 + tools	Long-horizon autonomous task completion
OpenAI Agents API	LLM + handoffs	Production multi-agent orchestration

Trend: Agentic AI = LLM planning + DRL execution + world model imagination

Part 4 — Application: Robotics

Why DRL for robotics?

- Continuous action spaces (joint torques) → SAC / TD3
- Sim-to-Real via domain randomization
- Hard to hand-engineer complex behaviors (dexterous manipulation)

SOTA 2025:

- **Isaac Lab (NVIDIA)**: 8,192 parallel envs → humanoid locomotion in hours
- **Diffusion Policy**: Multimodal action distributions → dexterous manipulation
- **OpenVLA / RT-2**: Language-conditioned robot control

Connection to Bonus 5:

LunarLanderContinuous is a direct analog: continuous thrust control → robot joint torques

Part 4 — Application: Game AI

The progression of game AI:

AlphaGo (2016) → AlphaZero (2018) → MuZero (2020) → Voyager (2023)
Human data → Self-play → World model → LLM + code

Key results:

- **AlphaStar**: Defeated world champion StarCraft II players (PPO + LSTM + league training)
- **OpenAI Five**: Dota 2 world champions (massive scale: 180 years/day self-play)
- **VPT (Minecraft)**: 70,000 hours of YouTube gameplay → general Minecraft agent

Insight: Scale (compute + data + self-play) consistently drives breakthroughs in game AI

Part 4 — Application: FinTech

Financial markets as MDP:

- State: Price series, fundamental data, market microstructure
- Action: Buy / Sell / Hold (discrete) or position size (continuous)
- Reward: Risk-adjusted return (Sharpe ratio)

Why DRL? Adapts to regime changes; discovers non-linear strategies; handles multi-asset correlation

FinRL benchmark results:

Algorithm	Annual Return	Sharpe Ratio
SAC	Best overall	1.8+
PPO	Stable	1.4
DQN	Moderate	1.2
Buy & Hold	Baseline	0.9

Key challenge: Non-stationarity + overfitting to historical data

Part 4 — Application: Autonomous Vehicles

Challenges unique to AV:

- Safety constraints (failure = injury)
- Multi-agent interaction with human drivers
- Rare but critical edge cases
- Real-time (<100ms) decisions

DRL approaches:

- **End-to-end:** Camera/LiDAR → steering/throttle (TransFuser)
- **Modular:** DRL for sub-components (planning, lane change)
- **Safe RL:** Constrained MDP enforcing safety during learning

Key simulators: CARLA (photorealistic), Highway-env (fast), MetaDrive (procedural)

WAYMO, Wayve, ApolloAuto all use hybrid approaches: DRL + rule-based + world models

Part 5 — SOTA Trends 2025–2026

1. World Models

Learn to *imagine* future states → plan without real-world interaction

DreamerV3: Single model achieves human-level on 150+ tasks (Atari, MuJoCo, Minecraft)

2. Diffusion Policy

Diffusion models for robot action distributions → captures multimodal behaviors

3. Embodied AI Foundation Models

OpenVLA, RT-2, π_0 — generalize across robot types and tasks

4. RL + Transformers

Decision Transformer, TransDreamer, ACT — attention for long-range dependencies

5. Offline RL → Online Fine-tuning

Pretrain on large offline datasets → fine-tune online with minimal real interaction

Part 6 — Comparative Analysis

Method	Strength	Weakness	Sample Efficiency	Real-world Use
DQN	Simple, Atari-proven	Discrete only, unstable	Low	Discrete games
PPO	Stable, RLHF standard	On-policy, data-inefficient	Medium	ChatGPT, robots
SAC	Best continuous control	Complex, slower	High	Robot manipulation
MuZero	Planning advantage	Very high compute	Very High	Board games
DT	Transformer scalability	No online improvement	N/A (offline)	Demonstrations

Our experiments confirm:

- PPO > DQN on CartPole (stability)
- SAC is necessary for continuous action spaces
- Hyperparameter sensitivity: $\text{lr}=3\text{e-}4$ optimal for PPO

Part 7 — Open Source Ecosystem

Repository	Stars	Best For
stable-baselines3	10k+	Research, education, prototyping
ray/rllib	35k+	Distributed, production-scale
cleanrl	5k+	Understanding algorithms (single-file)
spinningup	10k+	Learning DRL with math-code alignment
IsaacLab	Growing	GPU robot simulation
carla	12k+	Autonomous driving research
habitat-lab	2k+	Embodied AI, indoor navigation
FinRL	10k+	Algorithmic trading research
MineDojo	1.5k+	Open-ended embodied learning
ml-agents	17k+	Game environments, Unity

Bonus Experiments — Summary

5 Experiments, Complete Results

#	Algorithm	Environment	Result	Figure
1	DQN vs PPO	CartPole-v1	PPO: 500 / DQN: 144	✓
2	PPO × 3 envs	CartPole/MountainCar/Acrobot	475 / -120 / -100	✓
3	PPO ablation	CartPole (12 configs × 3 seeds)	Best: lr=3e-4, clip=0.1	✓
4	DQN GridWorld	10×10 custom maze	22 steps, 99% success	✓
5	SAC	LunarLanderContinuous-v3	286.8 ± 16.7 > 200	✓

Bonus 2 — PPO Multi-Environment

- **MountainCar** is hardest: sparse reward challenges on-policy PPO → motivates SAC's entropy exploration

Bonus 3 — Hyperparameter Ablation

- $\text{lr}=3\text{e-}4$ (PPO default) is robust across all clip values
- $\text{lr}=1\text{e-}4 + \text{clip}=0.4 \rightarrow$ unstable (only 335)

Bonus 4 — Custom GridWorld + DQN

- Q-table → Neural Network: same principle, continuous state space
- 22-step optimal path, 99% success rate after 4,848 episodes

Key Takeaways

1. Algorithm selection is task-specific

- Discrete action → DQN/PPO; Continuous → SAC/TD3

2. Agentic AI = DRL + LLM + World Models

- PPO for RLHF alignment; SAC for physical control

3. Scale is the most reliable driver of progress

- More compute + self-play + data = better results

4. Safety and alignment remain unsolved

- Reward hacking, distribution shift, sim-to-real gap are open problems

5. Foundation models are transforming DRL

- Pretrain offline → fine-tune online; generalize across tasks

Future Directions

- **World Models + LLM Planning** → General autonomous agents (AGI path)
- **Embodied AI** foundation models (RT-2, π_0) → Generalist robots
- **Offline RL pretraining** → Reduce real-world data needs
- **Safe RL** → Deploy DRL in safety-critical systems (AV, medical, finance)
- **Multi-agent emergence** → Cooperative intelligence at scale

Thank You

Experiments: All 5 bonus experiments successfully completed

- Code: `main.py` , `bonus_2~5_*.py`
- Results: 6 figures (PNG), 1 report (report.md)

References: 17 key papers from NeurIPS, ICML, ICLR, Nature (2015–2024)

Report: `report.md` / *Code:* *GitHub Repository* / *Slides:* `slides.md` (*Marp*)